Port scan e spoofing in una rete SDN

rilevamento, redirect su honeypot e quarantena con Ryu e Mininet

Campo	Valore
Studente	Francesco Sabatino, matricola DE9000191
Repository	https://github.com/giovannistanco/NCIS_unina/tree/main/2026/sabatino_francesco
Corso	Networks and Cloud Infrastructures, Prof. G. Ventre, A.A. 2025/2026

1. Obiettivo

Il progetto sfrutta la visione globale del controller SDN per difendere la rete da **port scan** e **spoofing** (ARP, MAC, IP). Il controller assegna a ogni host un **punteggio di rischio** che sale con i comportamenti sospetti e scende da solo nel tempo: in base al punteggio l'attaccante viene prima **deviato in modo trasparente su un honeypot** che si finge il server, poi, se continua, **messo in quarantena**; quando smette, le regole vengono tolte automaticamente. La risposta è applicata solo sulla porta dello switch dell'attaccante. La rete è emulata con Mininet e Open vSwitch, il controller è un'applicazione Ryu (OpenFlow 1.3).

1.1 Rispetto alla prima versione

Il progetto sostituisce quello consegnato in precedenza (ARP spoofing e port scan con quarantena), di cui corregge i limiti e realizza l'estensione proposta nelle sue conclusioni (redirect su honeypot con riscrittura degli header):

Prima versione	Questa versione
Learning switch con regole su porta e MAC di destinazione: dopo il primo pacchetto lo switch non interpella più il controller, quindi la scansione si vede solo se le sonde arrivano prima della regola	Regole fino alla porta di destinazione: ogni porta nuova genera un packet-in e la scansione è sempre visibile
Binding IP-MAC imparato dal primo ARP; quarantena sul MAC sorgente, aggirabile cambiando MAC	Inventario fidato dei binding; controllo di MAC e IP sorgente; quarantena sulla porta dello switch
Soglia fissa (8 porte in 5 s): una scansione lenta non viene vista	Punteggio con decadimento esponenziale
Solo drop, sblocco a tempo fisso con thread periodico	Redirect sull'honeypot e quarantena; sblocco quando l'host smette (idle timeout e FlowRemoved)
Topologia ad albero	Anello con ciclo, gestito senza flooding

2. Topologia

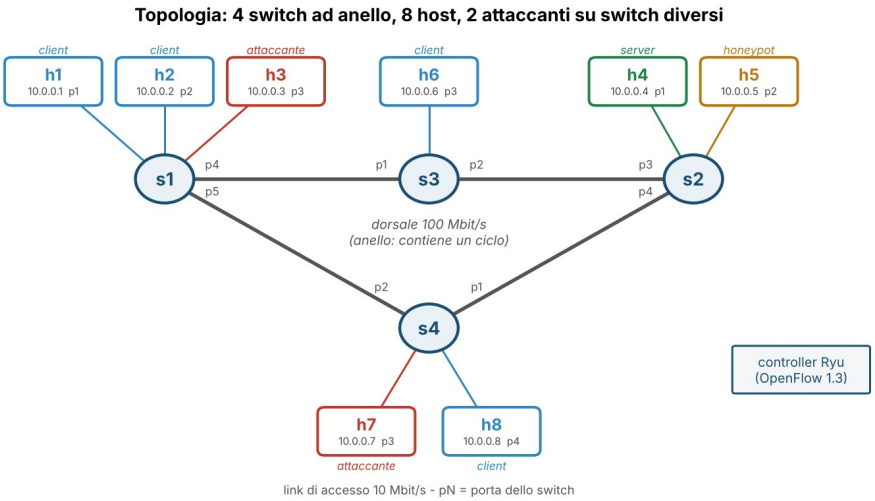


Figura 1. 4 switch ad anello e 8 host (pN = porta dello switch).

h3 e h7 sono gli attaccanti, su switch diversi e insieme a host legittimi; h4 è il server (web sulla porta 80, iperf sulla 5001); h5 è l'honeypot, con server web sulle porte 21, 22, 23, 80 e 8080 e una pagina diversa da quella di h4, così si vede subito se una richiesta è stata deviata. La rete contiene un ciclo: in una LAN tradizionale il flooding dei broadcast causerebbe una **tempesta di broadcast**, qui no, perché il controller non fa mai flooding.

3. Funzionamento del controller

3.1 Inventario, instradamento e regole

Il controller non usa il MAC learning: legge da un **inventario** (`network_config.py`) IP, MAC, switch e porta di ogni host, come il database del DHCP snooping di uno switch reale. All'avvio una **BFS** calcola per ogni switch la porta di uscita del percorso minimo verso ogni destinazione; le richieste ARP vanno sempre al controller, che risponde con il MAC registrato (**ARP proxy**). Si usa una sola tabella, con queste regole:

Priorità	Match	Azione
300	porta dell'host in quarantena	drop (idle timeout 30 s)
200	ARP	al controller (validazione e ARP proxy)
100	porta dell'attaccante, IP destinazione = server	destinazione riscritta verso h5 (idle timeout 30 s)
100	IP sorgente = honeypot, IP destinazione = attaccante	sorgente riscritta come h4
10	porta, MAC, IP, protocollo, porta di destinazione	inoltro sul percorso calcolato (idle timeout 10 s)
0	table-miss	al controller

Le regole di inoltro arrivano fino alla porta di destinazione, così ogni porta nuova contattata genera un packet-in e il controller vede la scansione. Le regole di sicurezza hanno priorità più alta, quindi valgono subito anche sui flussi già installati; la quarantena sta sopra l'ARP, così l'host bloccato non può mandare nemmeno ARP falsi.

3.2 Rilevamento e punteggio

I controlli si fanno solo sulle porte verso gli host (l'honeypot è escluso): la porta dello switch dice chi è davvero il mittente, MAC e IP dicono chi afferma di essere.

- **Spoofing** (+100): MAC sorgente diverso da quello registrato sulla porta, oppure MAC/IP dichiarati nell'ARP o IP sorgente diversi da quelli dell'host. Il pacchetto viene scartato.
- **Port scan** (+2): ogni SYN senza ACK verso una coppia (IP, porta) mai contattata prima dall'host. Ripetere una connessione o rispondere (SYN-ACK) non conta.

Il punteggio si dimezza ogni 30 s ($S = S_{prec} \cdot 2^{(-\Delta t/30)} + peso$). Soglie: **SUSPECT ≥ 12** (solo log), **REDIRECT ≥ 40**, **QUARANTINE ≥ 70**. Per una scansione di una porta nuova ogni T secondi il punteggio tende a $S_\infty = 2 / (1 - 2^{(-T/30)})$: 88 per T = 1 s, 44 per 2 s, 30 per 3 s, 10 per 10 s. Una scansione veloce arriva al redirect alla 21ª porta, uno spoofing va subito in quarantena.

3.3 Redirect e rilascio

Il **redirect** è un NAT su due regole: all'andata i pacchetti per il server vengono riscritti verso h5, al ritorno le risposte di h5 vengono riscritte come se arrivassero da h4. L'attaccante, che ha in cache il MAC di h4, non se ne accorge e continua contro un sistema che non ha niente da proteggere. Le sonde deviate non passano più dal controller, quindi una scansione del server si ferma a REDIRECT, mentre una scansione di altri host porta alla quarantena. I pacchetti già in coda verso il controller quando il livello cambia vengono trattati come farebbero le regole.

Per il **rilascio** le regole di redirect e quarantena hanno idle timeout di 30 s e il flag `OFPFF_SEND_FLOW_REM`: quando l'attaccante smette, lo switch rimuove la regola e manda un **FlowRemoved**. Il controller allora riporta l'host a NORMAL e cancella le sue regole rimaste, oppure, se il punteggio è ancora sopra la soglia (es. scansione lenta), reinstalla il blocco. Chi insiste resta bloccato senza che il controller interroghi gli switch.

4. Esecuzione

Requisiti: VM Ubuntu 22.04 del corso con Mininet 2.3 e Ryu (su Python 3.10 serve la correzione a `wsgi.py` indicata nelle slide), più `nmap`, `iperf`, `curl` e `dsniff`. Dalla cartella del repository:

```
ryu-manager controller.py        # terminale 1: controller
sudo python3 topology.py         # terminale 2: rete e servizi su h4 e h5
```

5. Esperimenti

Prove eseguite a mano dalla CLI di Mininet. Il controller stampa ogni cambio di livello con il tempo trascorso dal primo evento sospetto e, al rilascio, i pacchetti scartati o deviati. I SYN arrivati al server sono contati con una regola iptables su h4, quelli arrivati all'honeypot con tcpdump su h5.

Prova	Comando (CLI di Mininet)	Risultato misurato
Traffico legittimo	`pingall`, `h1 iperf -c 10.0.0.4 -p 5001 -t 10`	0% persi (56/56) nonostante il ciclo, 9,56 Mbit/s, nessun allarme
Scan del server	`h3 nmap -n -Pn -sS -T4 10.0.0.4`	REDIRECT in 0,06 s (0,06-0,07 s su tre prove); su 1000 porte, 20 SYN al server e 980 all'honeypot
Scan ripetuta	stesso comando, poi `h3 curl -s 10.0.0.4` e `h2 curl -s 10.0.0.4`	nmap vede aperte 21, 22, 23, 80, 8080 (su h4 solo 80 e 5001); h3 riceve la pagina dell'honeypot, h2 quella vera
Scan laterale	`h7 nmap -n -Pn -sS -T4 -p 1-100 10.0.0.1 10.0.0.2 10.0.0.6`	QUARANTINE in 0,09 s, porte filtered; ping di h8 (stesso switch) 0% persi
ARP spoofing	`h3 arpspoof -i h3-eth0 -t 10.0.0.4 10.0.0.1`	QUARANTINE al primo ARP falso; in h4 10.0.0.1 resta associato al MAC di h1
IP spoofing	`h2 nmap -n -Pn -sS -S 10.0.0.6 -e h2-eth0 --send-ip -p 80 10.0.0.4`	QUARANTINE immediata, SYN non consegnato (porta filtered)
Rilascio	30 s senza traffico dall'attaccante	NORMAL dopo 30 s (h7: 547 pacchetti scartati), nessuna regola residua

Tre osservazioni dalle prove. I 980 SYN arrivati all'honeypot sono 970 deviati dalla regola sullo switch e 10 deviati dal controller, perché erano già in coda verso di esso quando è scattato il redirect. La **prima** scansione vede quasi solo la porta 80 del server vero: nmap prova per prime le porte comuni (21, 22, 23, 80, 8080…), che rientrano nella ventina di sonde precedenti il redirect; ripetendola con h3 in REDIRECT si vedono i servizi dell'honeypot. Infine, con `-S` ma senza `--send-ip` nmap mette l'IP falso già nella richiesta ARP, e il controller blocca h2 come ARP spoofing prima ancora del SYN.

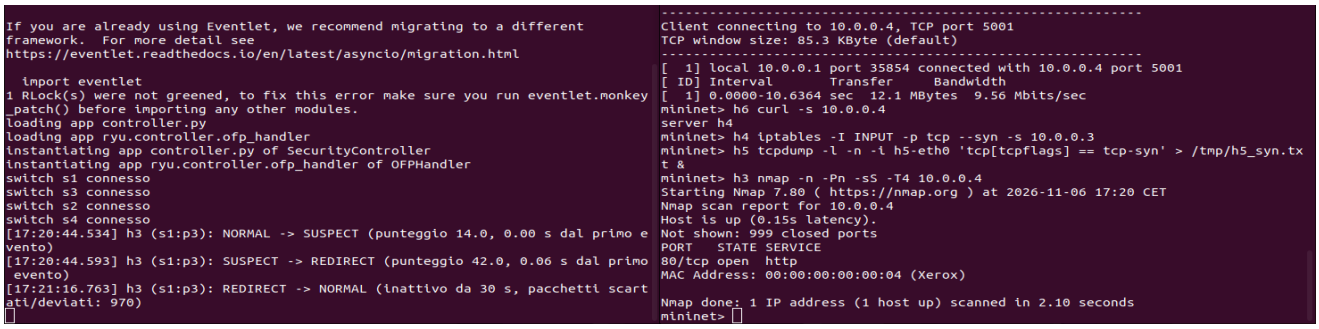


Figura 2. Prima scansione di h3: SUSPECT e REDIRECT in 0,06 s, rilascio dopo 30 s di inattività.

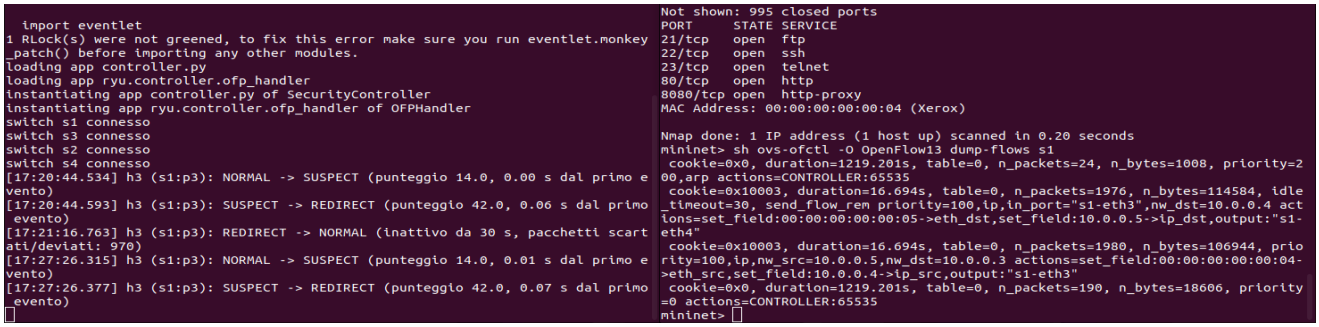


Figura 3. Regole su s1 durante il redirect: andata (destinazione riscritta verso h5) e ritorno (sorgente riscritta come h4).

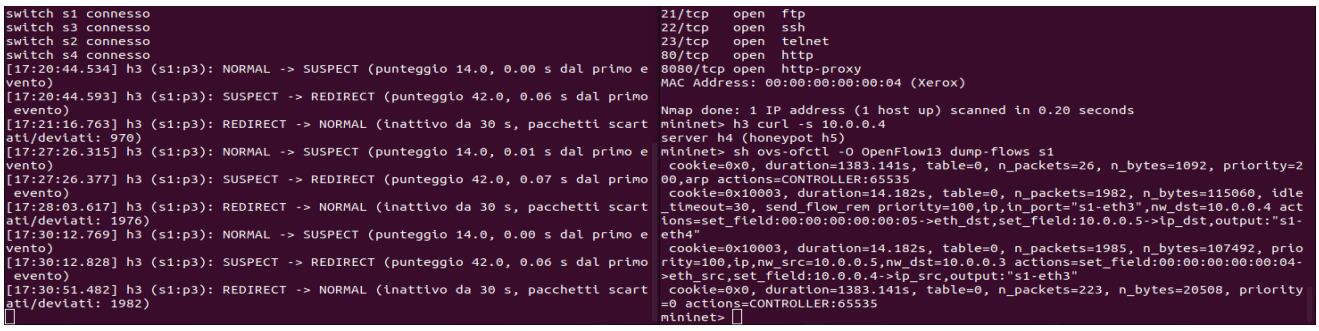


Figura 4. Scansione ripetuta: nmap vede le porte dell'honeypot e curl ne riceve la pagina, contattando sempre 10.0.0.4.

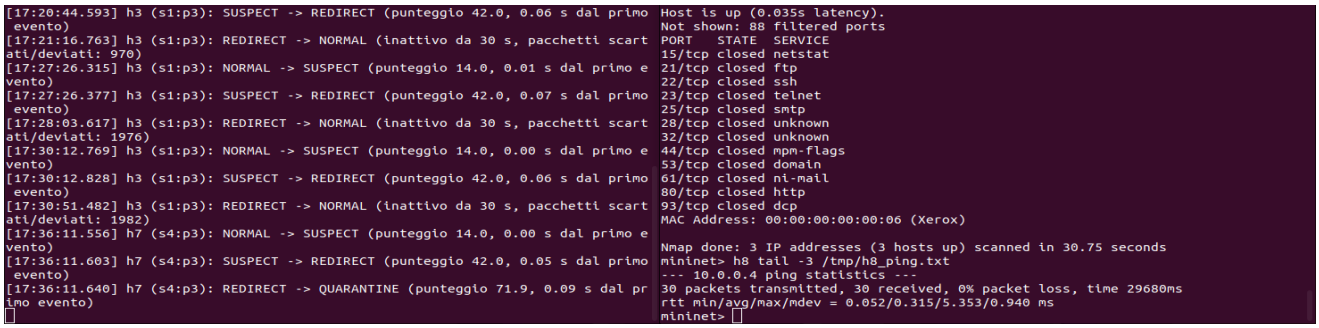


Figura 5. Scansione di altri host da parte di h7: QUARANTINE in 0,09 s; il ping di h8 non perde pacchetti.



Figura 6. ARP spoofing di h3: quarantena al primo ARP falso, la cache di h4 resta corretta.

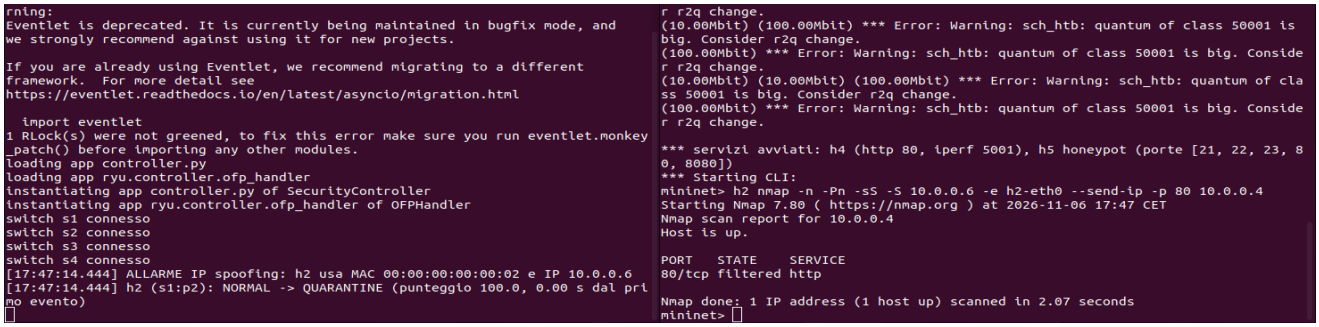


Figura 7. SYN con IP sorgente falso da h2: allarme di IP spoofing e quarantena; nmap non riceve risposta.

6. Limiti

- Inventario statico: in una rete reale i binding verrebbero dal DHCP snooping.
- Scansioni più lente di circa una porta ogni 2 s non arrivano al redirect, e quelle UDP non vengono contate.
- Prima del redirect una ventina di sonde raggiunge il server vero, e sono proprio quelle sulle porte più comuni.
- Honeypot a bassa interazione e soglie fisse; il primo pacchetto di ogni flusso passa dal controller, che con molti host diventerebbe un collo di bottiglia.

7. Conclusioni

Il sistema rileva port scan e spoofing e, in base a un punteggio di rischio, devia l'attaccante sull'honeypot oppure lo mette in quarantena, togliendo il blocco da solo quando l'host smette. Nelle prove il redirect è scattato in circa 0,06 s, solo 20 sonde su 1000 hanno raggiunto il server vero e l'attaccante, ripetendo la scansione, ha visto i servizi dell'honeypot come se fossero di h4, mentre gli host legittimi, anche sullo stesso switch dell'attaccante, non hanno subito nessun effetto. Tutto questo è possibile perché il controller SDN ha una visione globale della rete e programma gli switch con regole match + action: il redirect è un NAT ottenuto riscrivendo gli header e la quarantena è un semplice drop, entrambi installati via software solo sulla porta dell'attaccante.

Eventuali sviluppi futuri consistono:

- nell'utilizzare soglie diverse per ogni host, in modo da non segnalare erroneamente un host fidato che ha il compito di fare scansioni, come un host di monitoraggio;
- nel rendere più complesso l'honeypot, in modo che risulti più realistico e più difficile da identificare per l'attaccante.